

 UNIVERSIDAD DE ANTIOQUIA 1803	FACULTAD DE CIENCIAS EXACTAS Y NATURALES Física y Astronomía Primer Examen de Astrofísica Computacional NOMBRE: Melanie Andrea Muñoz Chavarria CEDULA: 1000758498
---	--

Informe desarrollo primer examen

En el siguiente informe se presentan los archivos, donde los que se encuentran en negrilla son los archivos principales que contienen el código pedido en cada punto. Y con una explicación del desarrollo y proceso que se realizó para cada literal.

1. Consulte y programe un algoritmo para encontrar el epsilon de máquina del computador donde normalmente programa.

Archivos:

- **epsilon de la maquina.c**: contiene el código comentado para el desarrollo de este literal

Desarrollo:

Se calculó el epsilon para tres tipos de variables: el float, double y long double. En particular estos tres tipos de variables, porque con ellas se almacenan números con decimales, es decir, que al ser sumados a un número, su suma no es entera, por lo que de alguna forma se presenta un error. Entonces dada la definición de epsilon de máquina, $1.0 + \epsilon\text{-mach} > 1.0$, buscando un epsilon hasta que $1.0 + \epsilon\text{-mach}$ sea aproximadamente uno.

Entonces primero se definieron las variables para cada epsilon y luego se les asignaron el valor de 1.0. Pues note que es el mayor valor decimal que puede tomar el epsilon sin alejarse demasiado del valor esperado, que en este caso es 1.0.

Luego se generaron los ciclos para obtener los epsilons. Esto se hizo dividiendo por dos el epsilon en cada ciclo, así se redujó el número a la mitad en cada iteración y se sumó de nuevo al 1.0, hasta obtener el valor esperado.

En particular, con el float y el long double, se hizo la conversión a float y a long double, de cada uno de los términos por los que se divide o se suma. Pues por defecto C, le asigna el valor de double.

En la impresión, se utilizó la cantidad de decimales para cada variable, a partir de los cuales se deja de visualizar decimales iguales a 0.

2. Elabore un programa que calcule π a partir de la siguiente fórmula

$$\sum_{n=0}^N \frac{(-1)^n}{2n+1} = \frac{\pi}{4}$$

Encuentre el valor de N mínimo que da como resultado un valor de pi con una precisión de 10 cifras decimales. Con el resultado del literal anterior, encuentre el número máximo de cifras decimales con el que su computador puede calcular π .

Archivos:

- **calculo_de_pi_con_formula.c** : contiene el código comentado para el desarrollo de este literal.

Desarrollo:

Para calcular un número pi con una precisión de 10 cifras decimales se tomó el valor del número pi, con un redondeo de diez cifras. Luego se aplicó un ciclo while, para calcular la serie de pi/4. El resultado del ciclo se multiplica por 4 para obtener el valor de pi.

En particular, el condicional del while es que el valor absoluto de la resta del pi con 10 cifras de redondeo menos el pi calculado con la serie. Esto tiene que ser mayor o igual al epsilon de la máquina, que para este caso fue el epsilon calculado anteriormente para la variable double.

Ahora, el número máximo de decimales que puede tener el número pi, basto con analizar el condicional del while:

$$|\pi - \pi_{serie}| \geq \epsilon_{máquina}$$

El máximo de esta diferencia se produce cuando el valor absoluto es igual al epsilon de la máquina

$$\max(|\pi - \pi_{serie}|) = \epsilon_{máquina}$$

Entonces la cantidad de decimales que tenga la diferencia, depende del epsilon de la máquina y su cantidad de decimales. Para saber la cantidad de decimales basta con aplicar logaritmo en base 10 al epsilon de la máquina, pues este se encuentra en potencias de diez. Para este caso se obtuvo que el valor máximo de decimales sería 16.

3. Realice un código para calcular el factorial de un número. Luego, use este código como una función para calcular la función de Bessel de orden α , $J_\alpha(x)$ usando la siguiente definición:

$$\sum_{k=0}^{\infty} \frac{(-1)^k}{k! (k+\alpha)!} \left(\frac{x}{2}\right)^{2k+\alpha}$$

Haga una gráfica con la función de Bessel de orden 0, 1, y 2 similar a la de la figura.

Archivos:

- **Funcion_Bessel.c** : contiene el código comentado con el desarrollo de este punto.
- **funcion_Bessel_alpha0.dat** : presenta dos columnas, en la primera los valores de x y en la segunda la evaluación de x en la función de Bessel. Para un alfa igual a 0.
- **funcion_Bessel_alpha1.dat** : presenta dos columnas, en la primera los valores de x y en la segunda la evaluación de x en la función de Bessel. Para un alfa igual a 1.
- **funcion_Bessel_alpha2.dat** : presenta dos columnas, en la primera los valores de x y en la segunda la evaluación de x en la función de Bessel. Para un alfa igual a 2.
- **grafica_funcion_Bessel.gp** : contiene el código para realizar el gráfico en gnuplot de los tres archivos anteriores.
- **funcion_bessel_para_diferentes_alphas.pdf** : contiene un posible resultado de correr el código anterior.

Desarrollo:

Lo primero que se hizo fue definir una función global para calcular el factorial. En esa función se usaron un iterador para recorrer desde uno hasta el número que se ingresó con un ciclo, y una variable factorial, donde se almacenó las multiplicaciones de cada ciclo, por lo que fue necesario inicializarlo con 1.0

En particular, este factorial se debió definir como un double, pues si bien el factorial está definido para números enteros, en las iteraciones del cálculo de la función de Bessel se produjeron errores, que no permitieron que se refleje el resultado de evaluar un x . Pues se presentaba que la operación a realizar no estaba definida y la serie no converge.

Luego se definió otra función para calcular la serie asociada a la función de Bessel, tomando un ciclo con una suma inicializada y con las condiciones de inicio, parada e incremento, dadas por la serie. A esta función se le ingresa el valor del límite superior de la serie, el valor de x y el valor de la variable alpha.

Luego, se creó un función basada en la función del código 7_rutinas_y_funciones.c, para poder crear un archivo donde se almacenaron los datos para cada x y su respectivo valor en la función de Bessel.

Después en la función principal, se probó la función factorial para diversos valores. Y se definieron los valores para llevar a cabo el cálculo de la función de Bessel. En particular,

cuando se toman límites superiores de la serie superiores a 150, la serie no converge para valores de x cada vez más grandes.

También se generó una entrada del usuario con el valor para α .

Y finalmente se iteró sobre un intervalo de valores de x entre 0 y 20, obteniendo cada uno de sus evaluaciones en la función de Bessel para cada valor de α , creando un archivo. Y esto fue graficado en gnuplot.

4. Consulte el método de integración de leap-frog y programarlo para repetir el experimento numérico de clase correspondiente a encontrar el número de puntos óptimo para hacer una simulación confiable del tiro parabólico. Luego, compare los resultados del experimento usando los dos integradores y concluya acerca de cuál es el mejor método de integración para hacer la simulación.

Archivos:

- **metodo_leap_frog_para_tiro_parabolico.c** : contiene el código comentado donde se desarrolla el movimiento parabólico con el método de leap frog y se analiza la conservación de energía y el error relativo de la misma. Y produce los siguiente archivos
- **parabolicoLeapFrog.dat** : datos de resultados de la solución del movimiento parabólico con el método de Leap-frog. En la primera columna el tiempo, en la segunda la velocidad en y . En la tercera la posición en x y en la última la posición en y .
- **conservacionEnergiaLeapFrog.dat** : en la primera columna están los datos del tiempo, en la segunda los del energía mecánica y en la tercera el error relativo
- **solucion_analitica_tiro_parabolico.c** : código que genera el siguiente archivo.
- **parabolico_analitico10000.dat** : contiene los resultados del movimiento parabólico para la solución analítica. En la primera columna el tiempo, en la segunda la velocidad en y . En la tercera la posición en x y en la última la posición en y .
- **parabolico_10000.dat** : contiene los resultados de la simulación del movimiento parabólico con el método de Euler. En la primera columna el tiempo, en la segunda la velocidad en y . En la tercera la posición en x y en la última la posición en y . Este archivo es un resultado del código 7_rutinas_y_funciones.c
- **analisis_energia_tiro_parabolico.c** : código que calcula la energía mecánica para cada tiempo en el método de Euler y produce el siguiente archivo
- **conservacion_energia10000.dat** : contiene los datos de energía para la simulación de Euler. En la primera fila contiene el tiempo, en la segunda la energía mecánica y en la segunda el error relativo.
- **energia_para_cada_simulacion.gp** : código para graficar la energía en función del tiempo, de las dos simulaciones, tanto para el método de Euler, como para el método de leap-frog.
- **grafico_para_cada_energia.pdf** : contiene un posible resultado de correr el código anterior.

Desarrollo:

Se crearon las variables globales para la aceleración de la gravedad y la velocidad inicial en x, que en este caso ambas no cambian.

Luego se definen 3 funciones: la primera función, se usó en el código 7_rutinas_y_funciones.c y esta se definió para crear el archivo con los datos del tiro parabólico. Luego se encuentra una función similar, pero para almacenar los valores de energía para cada tiempo y el error relativo, también para cada tiempo.

Y la última se utiliza para calcular la energía mecánica del sistema.

Luego se definieron las condiciones iniciales para el sistema, que en principio son las mismas que se utilizaron para el código de la solución analítica y el que realiza la simulación del tiro usando Euler. Y se asignó un valor a la velocidad inicial en x.

Además de que se crean otras variables para almacenar el tiempo máximo, número de puntos, delta de tiempo, tiempo. Y en particular se inicializaron las variables de posición en x, posición en y y velocidad en y, con los valores iniciales. Además, se guardó el valor de la energía mecánica inicial.

Estos valores iniciales se guardaron en la primera fila de los correspondientes archivos. Para después comenzar un ciclo donde se itera el tiempo, desde un valor ΔT hasta el tiempo máximo de simulación, con un crecimiento de ΔT .

En este ciclo se aplicó el método de integración de leap-frog, donde se calcula un valor de velocidad en y para un punto en el intermedio de un intervalo, para después evaluar la posición de x y y en este valor. Y finalmente, realizar un último paso, para la velocidad en y, tomando como una de sus componentes la velocidad en y para ese punto medio en un intervalo.

En particular, en la posición en x y la velocidad en x, no se evidencian estos cambios y la aplicación del método, pues la velocidad en x es constante.

Y finalmente para el valor de la velocidad de y se calculó el valor de la energía y el respectivo error relativo asociado a esta energía.

Después de ello, se genera una condicional para agregar todos los valores calculados en el ciclo, siempre y cuando sea mayor que cero.

El número de puntos tomado, es 10000, pues es el valor para el que el método de Euler estimó un error relativo menor a 0.1% en la energía mecánica. El método de leap-frog

presenta se acerca mucho más a la solución analítica, de hecho en la gráfica de energía mecánica se puede ver que se mantiene constante durante toda la simulación.

Además por la similitud de los archivos `parabolico_analitico10000.dat` y `parabolicoLeapFrog.dat` en particular en las columnas de datos de x y y , es fácil concluir que la integración con leap-frog logra simular acertadamente e incluso de forma superior a la de Euler, el tiro parabólico.

Se adjuntan algunos archivos con los códigos donde están contenidos, una gráfica del tiro parabólico para los tres datos que se tienen, las dos simulaciones y la solución analítica. Y los códigos donde se almacena la solución analítica y demás.

Referencias:

- *Floating point arithmetic and machine epsilon*. (s/f). Stack Overflow.
<https://stackoverflow.com/questions/16063820/floating-point-arithmetic-and-machine-epsilon>
- *Machine epsilon*. (s/f). Wikipedia. https://en.wikipedia.org/wiki/Machine_epsilon
- `7_rutinas_y_funciones.c`
- *Leapfrog integration*. (s/f). Wikipedia.
https://en.wikipedia.org/wiki/Leapfrog_integration